

Tytuł projektu: Fortuna – aplikacja dla klientów banku

Frontend: React, Vite, TypeScript, Redux Toolkit

Backend: ASP.NET Core, .NET, Entity Framework Core

Bazy danych: SQL Server, osobna baza zapisu i baza odczytu

Architektura: Clean Architecture, DDD, CQRS

Autor 1: Jakub Wołczko

Nr albumu: 70210

Autor 2: Jakub Łaszuk

Nr albumu: 72418

Repozytorium: [GitHub – Designing multi-tier business applications](#)

Spis treści

1	Cel i zakres systemu	4
2	Użyte technologie	4
2.1	Frontend	4
2.2	Backend	5
2.3	Bazy danych i narzędzia pomocnicze	5
3	Uruchomienie projektu	6
3.1	Wymagane narzędzia	6
3.2	Przygotowanie baz danych	6
3.3	Uruchomienie backendu	7
3.4	Uruchomienie frontendu	8
3.5	Proponowana ścieżka testowa po uruchomieniu	8
3.6	Typowe problemy konfiguracyjne	9
4	Widoki aplikacji	9
4.1	Landing page	9
4.2	Rejestracja użytkownika	10
4.3	Logowanie	11
4.4	Dashboard klienta	11
4.5	Przelewy	12
5	Architektura systemu	12
5.1	Warstwy backendu	13
5.2	CQRS i rozdzielenie zapisu od odczytu	15
6	Model domenowy	15
6.1	Agregaty i value objecty	16
6.2	Zdarzenia domenowe	16
7	Działanie backendu	16
7.1	Rejestracja i logowanie	16

7.2	Pobieranie dashboardu	17
7.3	Wykonywanie przelewu	18
7.4	Symulacja przelewu przychodzącego	19
8	Mechanizm Outbox i projekcje	19
9	Schematy baz danych	20
9.1	Baza zapisu	20
9.2	Baza odczytu	21
9.3	Najważniejsze relacje	21
10	Frontend	22
10.1	Struktura	22
10.2	Komunikacja z API	22
11	Bezpieczeństwo i walidacja	23
12	Dodatkowe diagramy architektoniczne	24
13	Podsumowanie	25

1 Cel i zakres systemu

Fortuna jest demonstracyjną aplikacją bankową dla klientów indywidualnych. System pozwala użytkownikowi założyć konto, zalogować się, obejrzeć dashboard produktów finansowych oraz wykonać przelew. Aplikacja została zaprojektowana jako system wielowarstwowy, w którym interfejs użytkownika komunikuje się z backendem przez REST API, a backend rozdziela model zapisu od modelu odczytu.

Zakres funkcjonalny obejmuje:

- stronę landing page z ofertą banku,
- rejestrację klienta oraz automatyczne utworzenie produktów startowych,
- logowanie z wykorzystaniem tokenu JWT,
- dashboard z podsumowaniem środków, listą produktów i historią zdarzeń,
- tworzenie przelewów własnych i zewnętrznych,
- symulację przelewu przychodzącego przez endpoint API,
- asynchroniczną aktualizację modelu odczytu przez mechanizm Outbox.

2 Użyte technologie

Projekt składa się z trzech głównych części: aplikacji frontendowej, backendu oraz skryptów baz danych. Technologie dobrano tak, aby pokazać typowy układ wielowarstwowej aplikacji biznesowej: klient WWW, API, warstwa przypadków użycia, model domenowy, infrastruktura i relacyjna baza danych.

2.1 Frontend

Technologia	Zastosowanie w projekcie
React 19	Budowa interfejsu użytkownika jako aplikacji SPA. Komponenty React odpowiadają za landing page, formularze logowania i rejestracji, dashboard oraz popup przelewu.
TypeScript	Typowanie kodu frontendu, kontraktów żądań i odpowiedzi API oraz struktur stanu aplikacji.
Vite	Serwer deweloperski i bundler aplikacji frontendowej. Uruchamia aplikację lokalnie pod adresem <code>http://localhost:5173</code> .
React Router	Routing po stronie klienta. Definiuje ścieżki <code>/</code> , <code>/login</code> , <code>/create-account</code> i <code>/dashboard</code> .
Redux Toolkit i React Redux	Przechowywanie stanu autoryzacji, tokenu JWT i informacji o sesji użytkownika.

Technologia	Zastosowanie w projekcie
TanStack Query	Pobieranie i cache'owanie danych dashboardu.
Axios / fetch wrapper	Komunikacja z backendem przez funkcję <code>apiRequest</code> .
CSS	Stylowanie widoków, popupów, przycisków, formularzy, watermarków i palety kolorów aplikacji.

2.2 Backend

Technologia	Zastosowanie w projekcie
.NET 10 / C#	Implementacja backendu, warstw aplikacyjnych, domeny i infrastruktury.
ASP.NET Core Web API	Udostępnienie endpointów REST dla rejestracji, logowania, dashboardu, kont i przelewów.
Entity Framework Core	Mapowanie modelu domenowego na tabele SQL Server oraz obsługa kontekstów <code>WriteDbContext</code> i <code>ReadDbContext</code> .
SQL Server provider	Dostęp do baz <code>FortunaWriteDb</code> i <code>FortunaReadDb</code> .
JWT Bearer Authentication	Autoryzacja endpointów chronionych. Token jest generowany po poprawnym logowaniu.
PBKDF2	Hashowanie i weryfikacja haseł klientów.
Swagger / OpenAPI	Dokumentacja i ręczne testowanie endpointów backendu z poziomu przeglądarki.
Hosted Service	Proces <code>OutboxMessageProcessor</code> działający w tle i przenoszący zdarzenia do modelu odczytu.

2.3 Bazy danych i narzędzia pomocnicze

Technologia / narzędzie	Rola
SQL Server 2025 Developer	Lokalny silnik baz danych używany przez backend.
SQL Server Management Studio	Uruchamianie skryptów <code>FortunaWriteDb.sql</code> i <code>FortunaReadDb.sql</code> .
Bruno	Ręczne testowanie endpointów API, np. przelewu przychodzącego.
PlantUML	Źródła diagramów UML w katalogu <code>docs/uml</code> .
LuaLaTeX	Kompilacja dokumentacji technicznej z pliku <code>documentation.tex</code> .

Technologia / narzędzie	Rola
Git	Kontrola wersji projektu.

3 Uruchomienie projektu

Do uruchomienia pełnego systemu potrzebne są trzy elementy: SQL Server z dwiema bazami danych, backend ASP.NET Core oraz frontend React. Zalecana kolejność to: przygotowanie baz danych, uruchomienie backendu, uruchomienie frontendu.

3.1 Wymagane narzędzia

- Visual Studio 2026 albo JetBrains Rider do uruchamiania backendu.
- .NET 10 SDK.
- Node.js w wersji 25.8.1 lub nowszej oraz npm.
- SQL Server 2025 Developer.
- SQL Server Management Studio albo inne narzędzie do wykonywania skryptów T-SQL.
- Visual Studio Code do wygodnej pracy z frontendem.
- Bruno do ręcznego testowania API.
- Git do pobrania repozytorium.

Po instalacji Node.js warto sprawdzić, czy programy zostały dodane do zmiennej PATH:

```
node --version
npm --version
```

Analogicznie można sprawdzić instalację .NET:

```
dotnet --version
```

3.2 Przygotowanie baz danych

Backend korzysta z dwóch baz danych:

- **FortunaWriteDb** – baza zapisu, przechowująca model transakcyjny,
- **FortunaReadDb** – baza odczytu, przechowująca projekcje dashboardu.

Kroki uruchomienia baz:

1. Uruchomić SQL Server.
2. Otworzyć SQL Server Management Studio.

3. Połączyć się z lokalną instancją SQL Server, np. `localhost`.
4. Otworzyć projekt bazodanowy `database/FortunaDb.slnx` albo bezpośrednio otworzyć skrypty SQL.
5. Wykonać skrypt `database/FortunaWriteDb.sql`.
6. Wykonać skrypt `database/FortunaReadDb.sql`.

Skrypty tworzą bazy, tabele, indeksy i wymagane relacje. Są napisane tak, aby mogły być uruchomione na pustej lokalnej instancji SQL Server. Domyślna konfiguracja backendu zakłada połączenie:

```
Server=localhost;Database=FortunaWriteDb;  
Trusted_Connection=True;TrustServerCertificate=True
```

```
Server=localhost;Database=FortunaReadDb;  
Trusted_Connection=True;TrustServerCertificate=True
```

Jeśli baza działa na innej instancji lub wymaga logowania SQL, należy zmienić wartości w pliku `backend/src/Fortuna.Api/appsettings.json`.

3.3 Uruchomienie backendu

Backend znajduje się w katalogu `backend`. Najwygodniejszy sposób uruchomienia:

1. Otworzyć plik `backend/Fortuna.slnx` w Visual Studio albo Riderze.
2. Ustawić projekt startowy `Fortuna.Api`.
3. Sprawdzić konfigurację `appsettings.json`, szczególnie sekcje `Database`, `Jwt` i `Cors`.
4. Uruchomić projekt.

Backend można także uruchomić z terminala. Z katalogu głównego repozytorium należy przejść do projektu API, odtworzyć zależności i uruchomić aplikację:

```
cd backend/src/Fortuna.Api  
dotnet restore  
dotnet run
```

Po uruchomieniu terminal wyświetli adres, na którym nasłuchuje aplikacja, np. `https://localhost:xxxx` albo `http://localhost:xxxx`. Swagger UI jest dostępny pod adresem:

```
https://localhost:xxxx/swagger  
http://localhost:xxxx/swagger
```

Endpoint / przekierowuje do `/swagger`. Dostępny jest również prosty endpoint kontrolny:

GET /api/health

Przed uruchomieniem backendu z terminala należy upewnić się, że SQL Server działa oraz że wykonano skrypty tworzące bazy `FortunaWriteDb` i `FortunaReadDb`. W przeciwnym razie API uruchomi się, ale operacje korzystające z bazy danych zakończą się błędem połączenia albo błędem braku tabel.

Sekcja CORS w konfiguracji dopuszcza frontend działający lokalnie pod adresem `http://localhost:5173`. Jeżeli frontend zostanie uruchomiony na innym porcie, trzeba dodać ten adres do `Cors:AllowedOrigins`.

3.4 Uruchomienie frontendu

Frontend znajduje się w katalogu `frontend`. Kroki uruchomienia:

```
cd frontend
npm install
npm run dev
```

Po uruchomieniu Vite aplikacja jest dostępna pod adresem:

`http://localhost:5173`

Najważniejsze skrypty npm:

Komenda	Opis
<code>npm run dev</code>	Uruchamia lokalny serwer deweloperski Vite.
<code>npm run build</code>	Kompiluje TypeScript i tworzy produkcyjny build aplikacji.
<code>npm run lint</code>	Uruchamia ESLint dla kodu frontendu.
<code>npm run preview</code>	Uruchamia podgląd produkcyjnego builda.

3.5 Proponowana ścieżka testowa po uruchomieniu

Po uruchomieniu baz, backendu i frontendu można sprawdzić działanie aplikacji w następujący sposób:

1. Wejść na `http://localhost:5173`.
2. Otworzyć formularz rejestracji i utworzyć użytkownika.
3. Zalogować się utworzonym adresem e-mail i hasłem.
4. Po zalogowaniu przejść na dashboard.
5. Wykonać przelew z poziomu popupu przelewów.
6. Sprawdzić, czy saldo i zdarzenia na dashboardzie są odświeżane.
7. Opcjonalnie użyć Bruno albo Swaggera do wywołania `POST /api/transfers/incoming`, aby zasymulować przelew przychodzący.

3.6 Typowe problemy konfiguracyjne

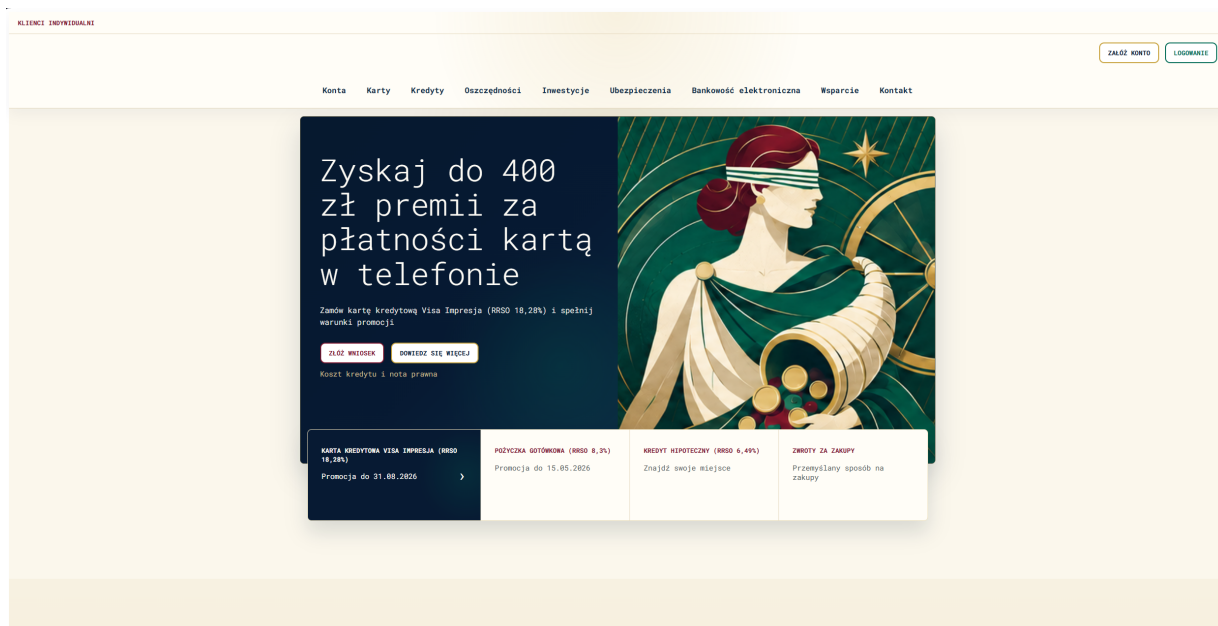
- Jeżeli backend nie łączy się z bazą, należy sprawdzić, czy SQL Server działa oraz czy connection stringi w `appsettings.json` wskazują poprawną instancję.
- Jeżeli frontend zgłasza błąd CORS, należy upewnić się, że adres frontendu znajduje się w sekcji `Cors:AllowedOrigins`.
- Jeżeli logowanie działa, ale dashboard zwraca błąd autoryzacji, należy sprawdzić, czy token JWT jest dołączany w nagłówku `Authorization`.
- Jeżeli dashboard nie pokazuje najnowszych danych od razu po operacji, należy pamiętać, że model odczytu jest aktualizowany asynchronicznie przez Outbox processor.
- Jeżeli token wygasa w trakcie testów, można ponownie się zalogować. Domyślny czas życia tokenu w konfiguracji wynosi 5 minut.

4 Widoki aplikacji

Frontend jest aplikacją SPA napisaną w React. Routing definiuje trzy ścieżki publiczne: landing page, logowanie i rejestrację oraz ścieżkę chronioną dashboardu. Po zalogowaniu token i identyfikator klienta są przechowywane po stronie klienta i używane do autoryzowanych wywołań API.

4.1 Landing page

Landing page prezentuje ofertę promocyjną banku, główną grafikę nawiązującą do Fortuny oraz kafle ofertowe. Z poziomu tego widoku użytkownik może przejść do rejestracji albo otworzyć modal logowania.



Rysunek 1: Landing page aplikacji Fortuna.

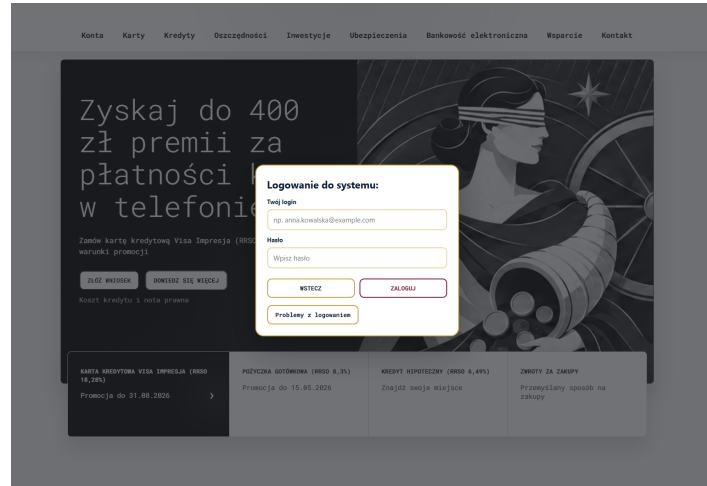
4.2 Rejestracja użytkownika

Formularz rejestracji zbiera imię, nazwisko, adres e-mail, hasło oraz typ klienta. Po przesłaniu formularza frontend wywołuje endpoint `POST /api/customers`. Backend waliduje dane, hashuje hasło i tworzy klienta razem z domyślnymi produktami.

Rysunek 2: Ekran rejestracji klienta.

4.3 Logowanie

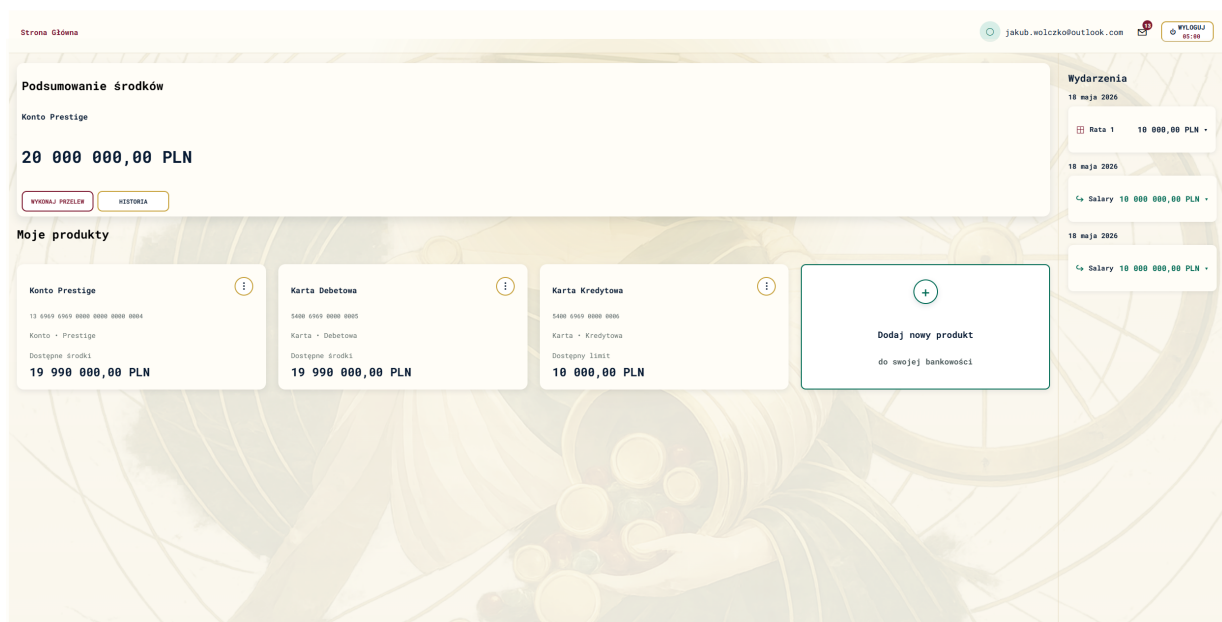
Logowanie odbywa się przez endpoint `POST /api/customers/login`. W odpowiedzi API zwraca token JWT oraz datę wygaśnięcia. Token jest następnie dołączany w nagłówku `Authorization: Bearer ...` do żądań dashboardu i przelewów.



Rysunek 3: Popup logowania.

4.4 Dashboard klienta

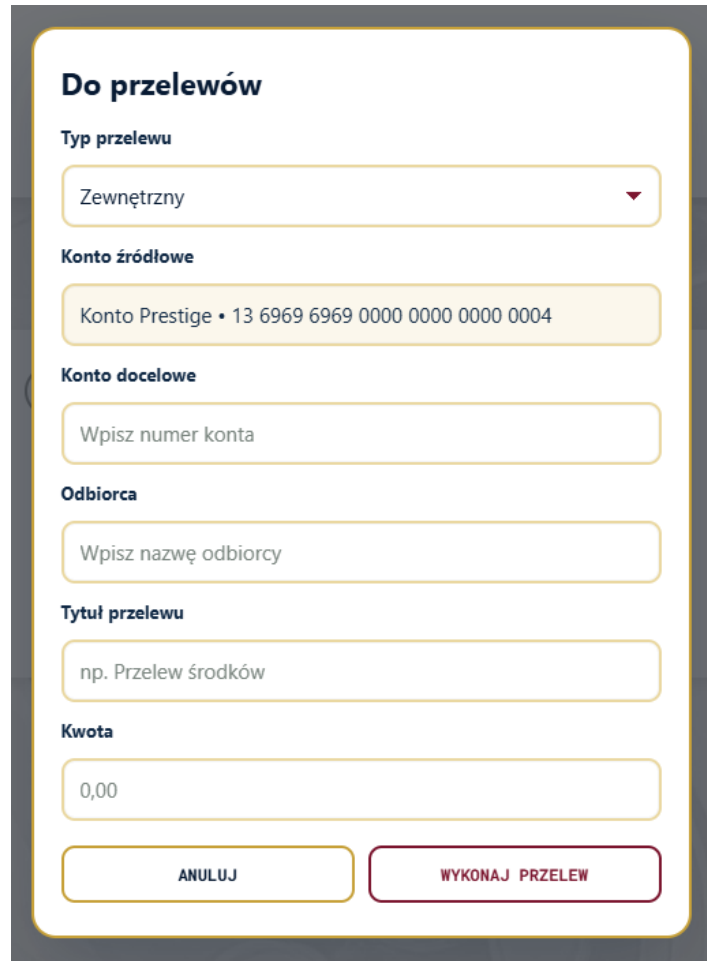
Dashboard pokazuje łączną wartość środków, produkty klienta, skrót zdarzeń oraz akcje bankowe. Dane są pobierane z endpointu `GET /api/dashboard/{customerId}`, który korzysta z osobnego modelu odczytu.



Rysunek 4: Dashboard klienta po zalogowaniu.

4.5 Przelewy

Popup przelewu pozwala wybrać typ przelewu, konto źródłowe, odbiorcę, tytuł i kwotę. Dla przelewów własnych wymagany jest produkt docelowy klienta, a dla przelewów zewnętrznych numer konta i nazwa odbiorcy.



Do przelewów

Typ przelewu

Zewnętrzny ▼

Konto źródłowe

Konto Prestige • 13 6969 6969 0000 0000 0004

Konto docelowe

Wpisz numer konta

Odbiorca

Wpisz nazwę odbiorcy

Tytuł przelewu

np. Przelew środków

Kwota

0,00

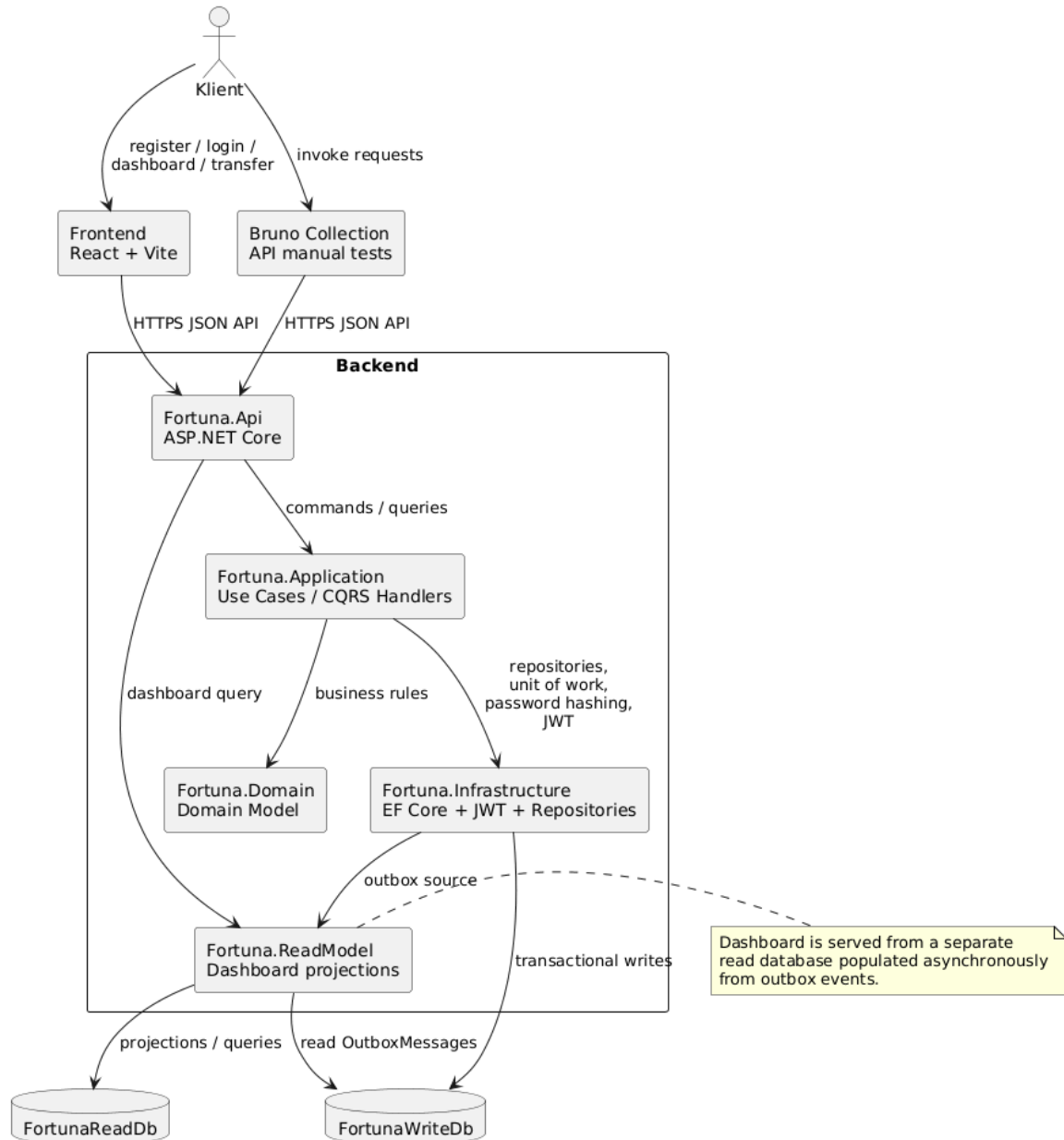
ANULUJ WYKONAJ PRZELEW

Rysunek 5: Formularz wykonywania przelewu.

5 Architektura systemu

System składa się z aplikacji frontendowej, backendu ASP.NET Core oraz dwóch logicznych modeli danych: zapisu i odczytu. Backend jest podzielony na projekty odpowiadające warstwom Clean Architecture.

Fortuna - System Context

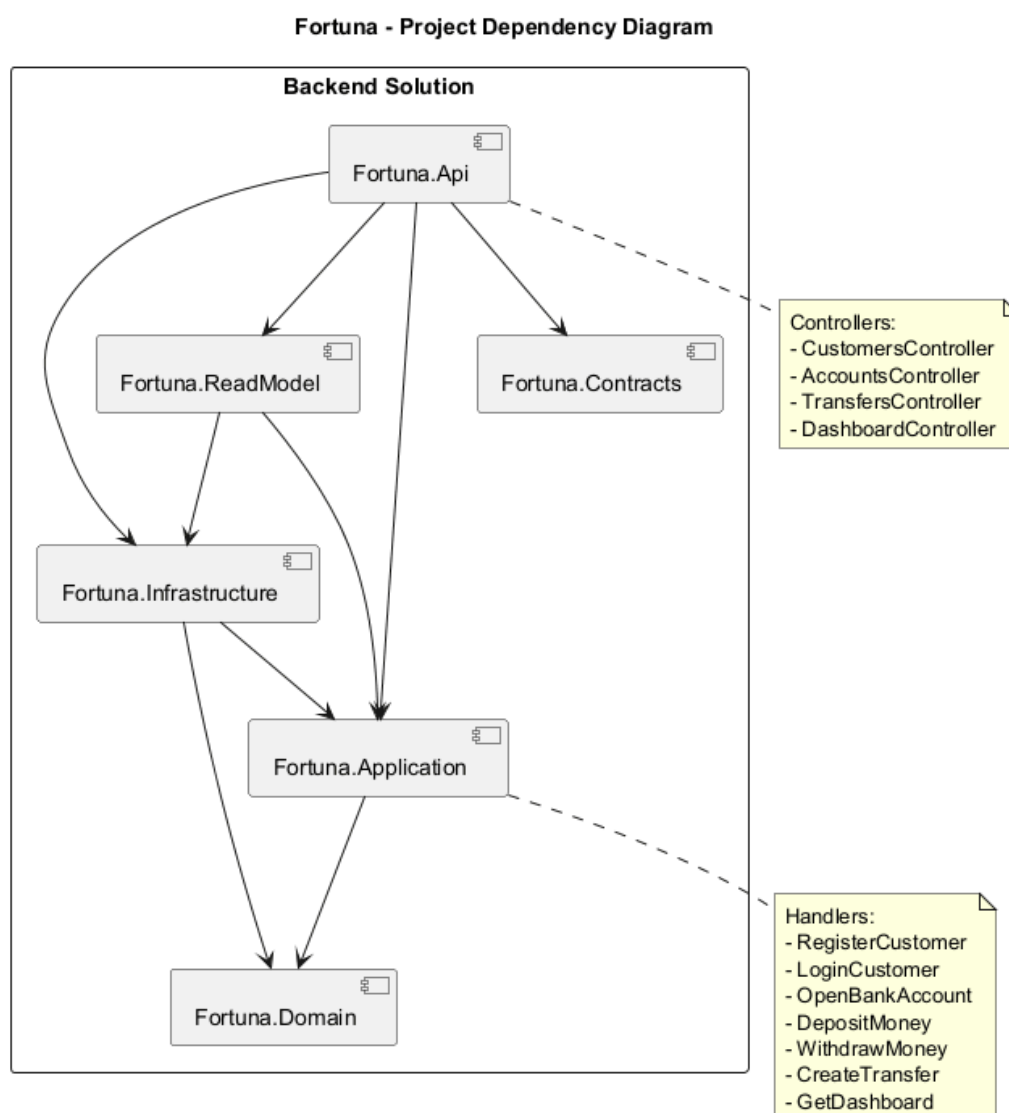


Rysunek 6: Kontekst systemu Fortuna.

5.1 Warstwy backendu

Projekt	Odpowiedzialność
Fortuna.Api	Warstwa HTTP. Zawiera kontrolery, konfigurację CORS, Swagger, JWT, middleware obsługi wyjątków i mapowanie endpointów.
Fortuna.Application	Przypadki użycia systemu. Zawiera komendy, zapytania, handlers CQRS, abstrakcje repozytoriów, walidację scenariuszy i DTO dashboardu.

Projekt	Odpowiedzialność
Fortuna.Domain	Model domenowy DDD: agregaty, encje, value objecty, reguły biznesowe i zdarzenia domenowe.
Fortuna.Infrastructure	Implementacje techniczne: EF Core dla bazy zapisu, repozytoria, Unit of Work, hashowanie haseł, JWT, Outbox processor.
Fortuna.ReadModel	Baza odczytu dashboardu, projekcje zdarzeń domenowych i repozytorium zapytań dashboardu.
Fortuna.Contracts	Kontrakty API używane przez kontrolery: requesty i response'y.



Rysunek 7: Zależności projektów backendu.

Zależności są skierowane do wewnątrz. Warstwa domenowa nie zależy od infrastruktury ani API. Warstwa aplikacyjna zna domenę i abstrakcje, ale nie zna szczegółów EF Core czy JWT. Infrastruktura implementuje interfejsy wymagane przez Application.

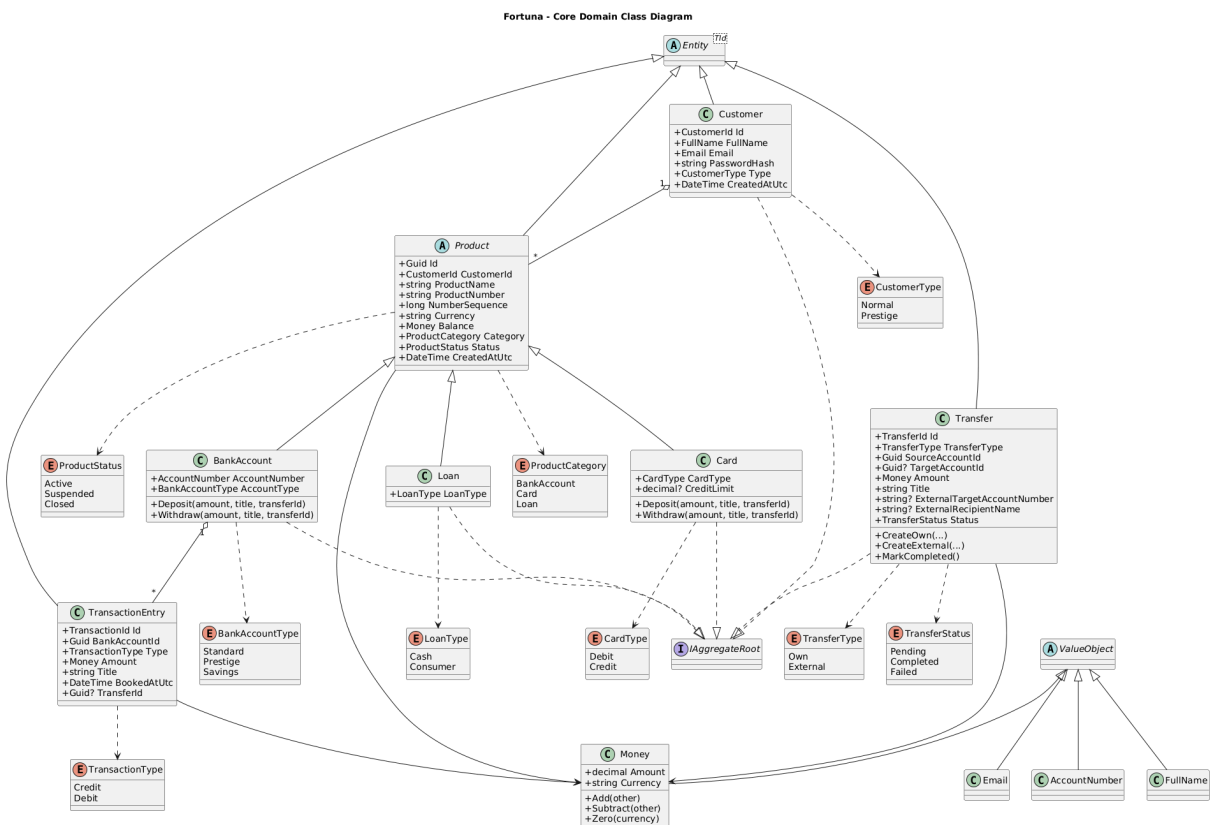
5.2 CQRS i rozdzielenie zapisu od odczytu

Operacje zmieniające stan systemu są realizowane przez komendy, np. `RegisterCustomerCommand`, `CreateTransferCommand`, `DepositMoneyCommand`. Operacje odczytowe są realizowane przez zapytania, np. `GetDashboardQuery`.

Dashboard nie czyta bezpośrednio tabel transakcyjnych. Zamiast tego korzysta z bazy odczytu `FortunaReadDb`, która zawiera gotowe widoki danych: kafle produktów oraz zdarzenia osi czasu. Dzięki temu odczyt dashboardu jest prosty i szybki, a model zapisu może pozostać bliższy regułom domenowym.

6 Model domenowy

Główne pojęcia domenowe to klient, produkty finansowe, pieniądze, transakcje i przelewy. Produkty finansowe mają wspólną klasę bazową `Product`, po której dziedziczą `BankAccount`, `Card` i `Loan`. Klient jest właścicielem produktów, a konto lub karta mogą rejestrować operacje wpłat i wypłat.



Rysunek 8: Diagram klas domenowych.

6.1 Agregaty i value objecty

Model używa podejścia DDD. Agregatami są między innymi `Customer`, `BankAccount`, `Card`, `Loan` i `Transfer`. Value objecty reprezentują dane z walidacją i znaczeniem domenowym, np. `Money`, `Email`, `FullName`, `AccountNumber`.

- `Money` przechowuje kwotę i walutę oraz udostępnia operacje dodawania i odejmowania.
- `Email` reprezentuje adres e-mail klienta i jest używany przy rejestracji oraz logowaniu.
- `FullName` rozdziela imię i nazwisko klienta.
- `AccountNumber` reprezentuje numer rachunku bankowego.

6.2 Zdarzenia domenowe

Encje domenowe generują zdarzenia, które są następnie zapisywane w tabeli `OutboxMessages`. Przykładowe zdarzenia to:

- `ProductCreatedDomainEvent`,
- `MoneyDepositedDomainEvent`,
- `MoneyWithdrawnDomainEvent`,
- `TransferCreatedDomainEvent`,
- `TransferCompletedDomainEvent`.

Zdarzenia są podstawą aktualizacji modelu odczytu.

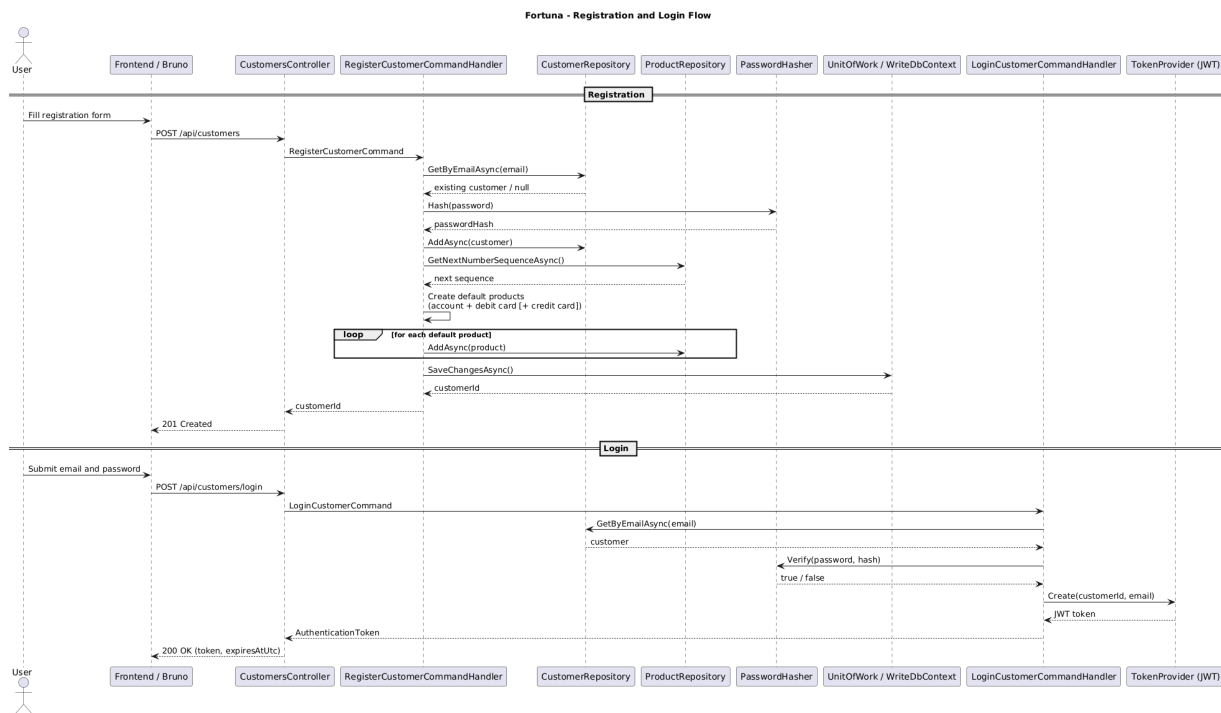
7 Działanie backendu

7.1 Rejestracja i logowanie

Rejestracja jest obsługiwana przez `CustomersController` i `RegisterCustomerCommandHandler`. Handler sprawdza długość hasła, waliduje unikalność e-maila, hashuje hasło algorytmem PBKDF2 i zapisuje nowego klienta. Następnie tworzy produkty startowe:

- konto standardowe albo prestiżowe,
- kartę debetową,
- dodatkową kartę kredytową dla klienta typu `Prestige`.

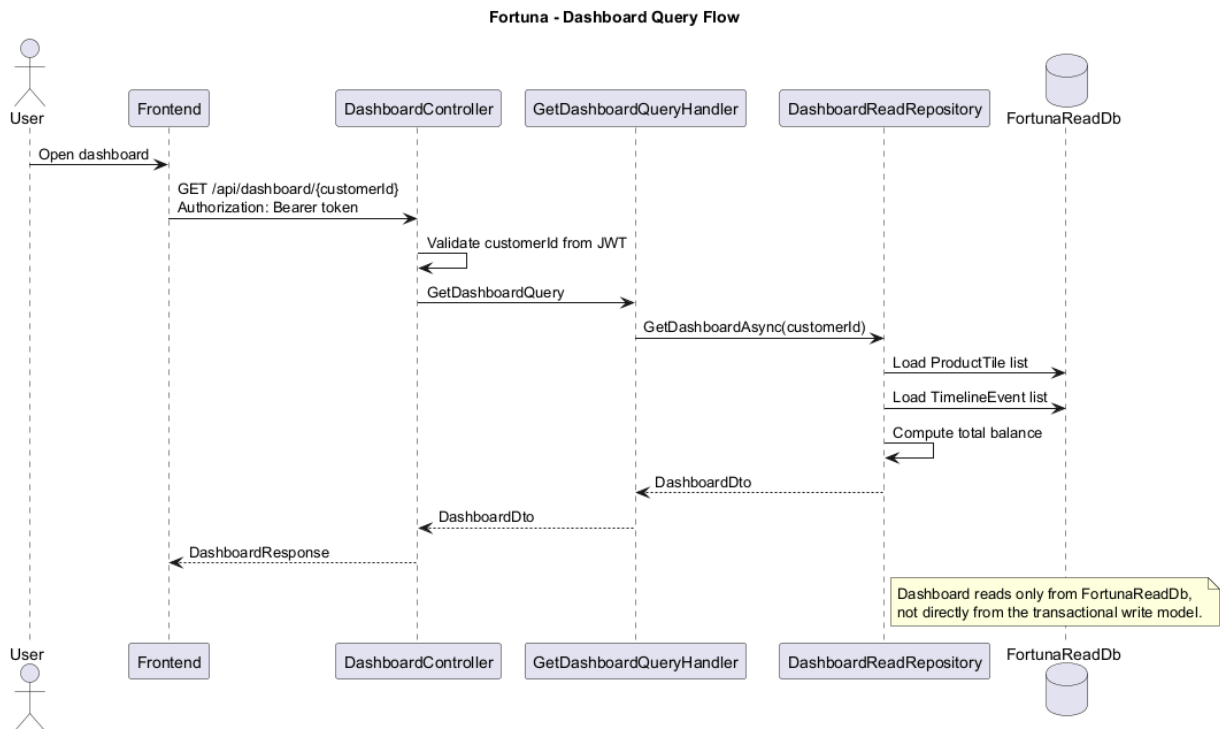
Logowanie jest obsługiwane przez `LoginCustomerCommandHandler`. Handler pobiera klienta po adresie e-mail, sprawdza hasło i generuje token JWT. Token zawiera identyfikator klienta, który później służy do autoryzacji zapytań.



Rysunek 9: Przepływ rejestracji i logowania.

7.2 Pobieranie dashboardu

Frontend wysyła żądanie `GET /api/dashboard/{customerId}` z tokenem JWT. Kontroler porównuje identyfikator z adresu URL z identyfikatorem zapisanym w tokenie. Jeśli identyfikatory są różne, zwracany jest `Forbidden`. W przeciwnym razie `GetDashboardQueryHandler` pobiera dane z `DashboardReadRepository`.



Rysunek 10: Przepływ pobierania dashboardu.

7.3 Wykonywanie przelewu

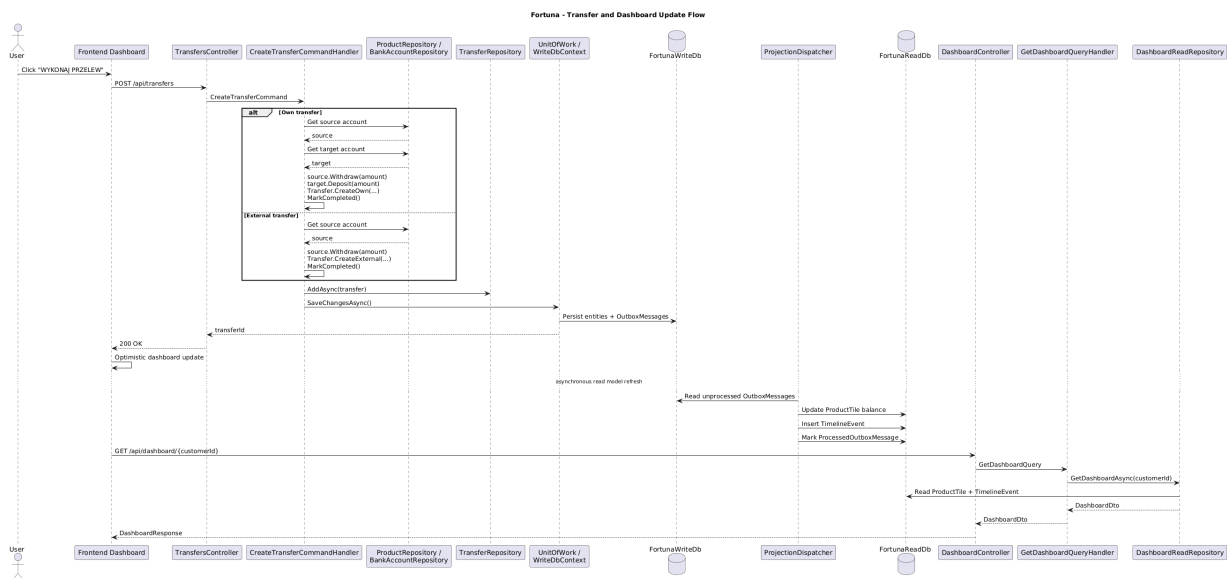
Przelew jest obsługiwany przez `TransfersController` i `CreateTransferCommandHandler`. Handler odczytuje produkt źródłowy i sprawdza, czy należy do zalogowanego klienta. Następnie rozróżnia przelew własny i zewnętrzny.

Dla przelewu własnego system:

- pobiera produkt źródłowy i docelowy,
- sprawdza, czy oba należą do klienta,
- wykonuje wypłatę ze źródła,
- wykonuje wpłatę na cel,
- zapisuje przelew i oznacza go jako zakończony.

Dla przelewu zewnętrznego system:

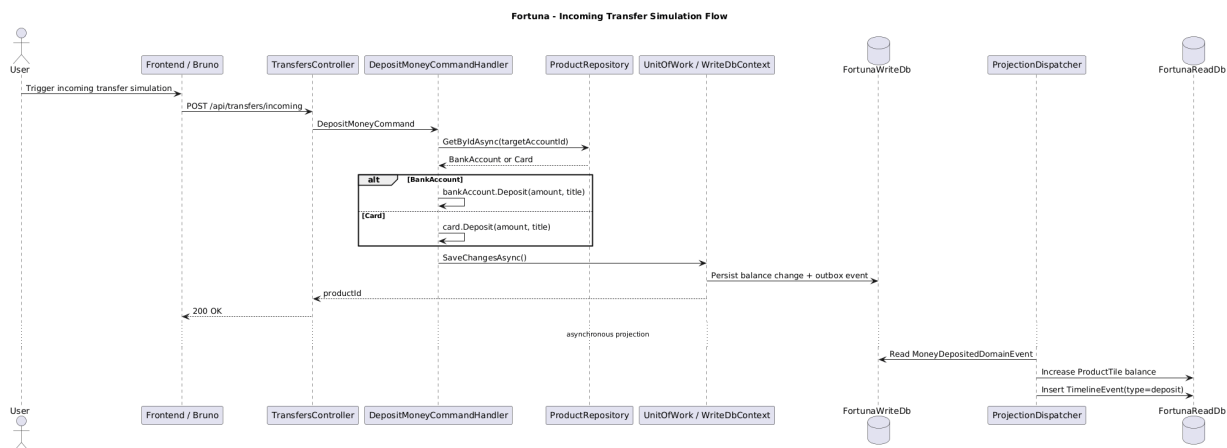
- wymaga numeru rachunku odbiorcy i nazwy odbiorcy,
- wykonuje wypłatę z produktu źródłowego,
- zapisuje przelew zewnętrzny,
- oznacza przelew jako zakończony.



Rysunek 11: Przelew i aktualizacja dashboardu.

7.4 Symulacja przelewu przychodzącego

Endpoint POST /api/transfers/incoming jest oznaczony jako AllowAnonymous. Służy do symulacji zewnętrznego wpływu środków, np. przy testowaniu przez Bruno. Komenda DepositMoneyCommand pobiera produkt docelowy, sprawdza, czy obsługuje wpłatę, zwiększa saldo i zapisuje zmiany.



Rysunek 12: Symulacja przelewu przychodzącego.

8 Mechanizm Outbox i projekcje

Podczas SaveChangesAsync w WriteDbContext system przegląda encje posiadające zdarzenia domenowe. Każde zdarzenie jest serializowane do JSON i zapisywane jako rekord w tabeli OutboxMessages. Po zapisaniu zmiany zdarzenia są czyszczone z encji.

Proces w tle OutboxMessageProcessor cyklicznie pobiera nieprzetworzone wiadomości z bazy

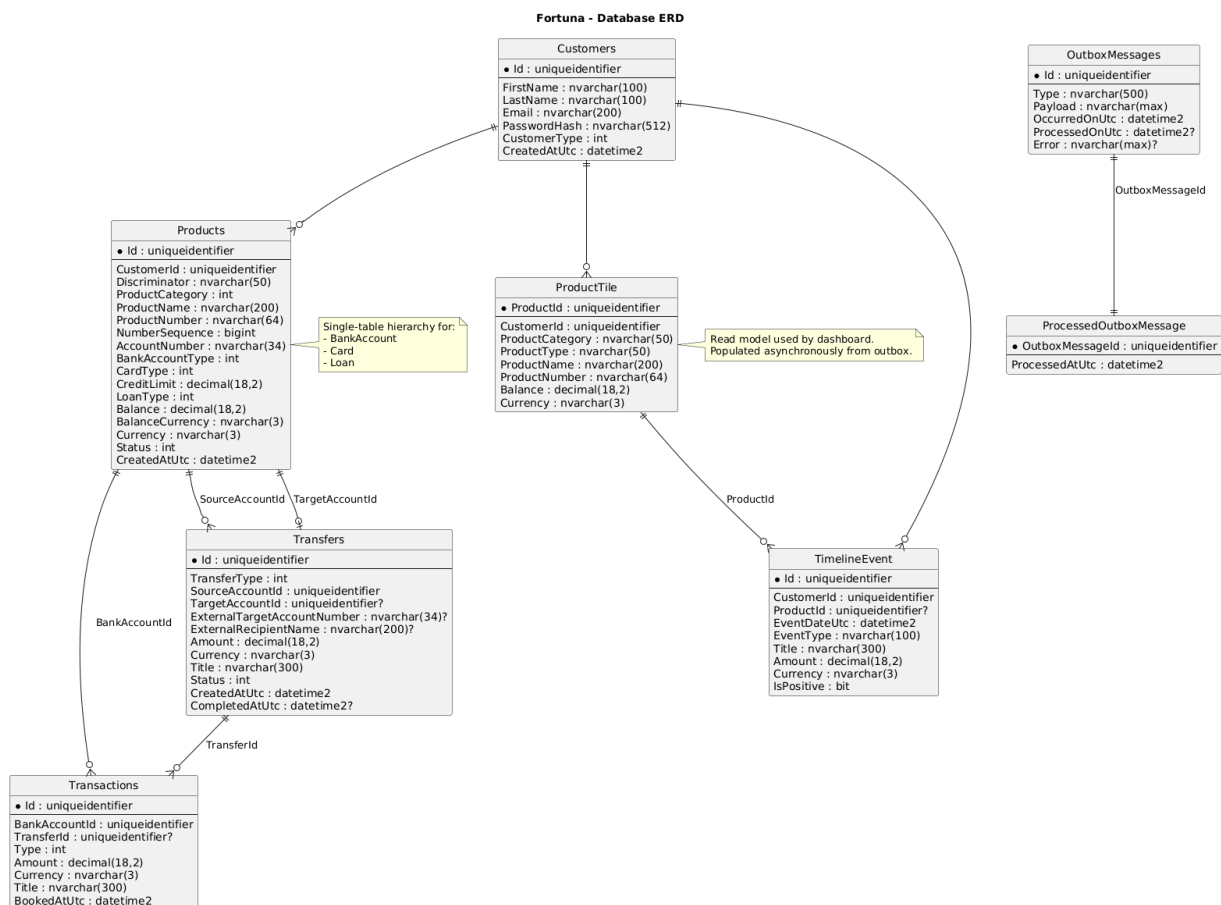
zapisu. Dla każdej wiadomości wywoływany jest `ProjectionDispatcher`, który aktualizuje bazę odczytu. Po poprawnym przetworzeniu rekord w `OutboxMessages` otrzymuje `ProcessedOnUtc`, a w bazie odczytu zapisywany jest wpis `ProcessedOutboxMessage`.

Projekcje obsługują między innymi:

- utworzenie kafła produktu po `ProductCreatedDomainEvent`,
- zwiększenie salda i dodanie zdarzenia osi czasu po `MoneyDepositedDomainEvent`,
- zmniejszenie salda i dodanie zdarzenia osi czasu po `MoneyWithdrawnDomainEvent`.

9 Schematy baz danych

System używa dwóch logicznych baz danych SQL Server: bazy zapisu `FortunaWriteDb` i bazy odczytu `FortunaReadDb`. Schemat jest pokazany na diagramie ERD.



Rysunek 13: Diagram ERD baz danych.

9.1 Baza zapisu

Baza zapisu przechowuje stan transakcyjny systemu i jest zarządzana przez `WriteDbContext`. To ona jest źródłem prawdy dla operacji domenowych.

Tabela	Opis
Customers	Dane klientów: identyfikator, imię, nazwisko, e-mail, hash hasła, typ klienta i data utworzenia. E-mail jest unikalny.
Products	Wspólna tabela dla produktów finansowych. EF Core używa dziedziczenia typu single-table hierarchy z kolumną Discriminator . W tabeli znajdują się konta, karty i kredyty.
Transactions	Zapisy księgowe dla wpłat i wypłat. Zawierają produkt, typ transakcji, kwotę, walutę, tytuł, datę księgowania i opcjonalny identyfikator przelewu.
Transfers	Przelewy własne i zewnętrzne. Zawierają źródło, opcjonalny cel wewnętrzny, dane odbiorcy zewnętrznego, kwotę, status oraz daty utworzenia i zakończenia.
OutboxMessages	Zserializowane zdarzenia domenowe oczekujące na projekcję do bazy odczytu.

9.2 Baza odczytu

Baza odczytu jest zoptymalizowana pod dashboard. Nie odtwarza pełnego modelu domenowego, tylko gotowe struktury potrzebne do wyświetlenia interfejsu.

Tabela	Opis
ProductTile	Kafle produktów widoczne na dashboardzie. Zawiera produkt, klienta, kategorię, typ, nazwę, numer, saldo i walutę.
TimelineEvent	Zdarzenia widoczne w panelu aktywności. Zawiera klienta, opcjonalny produkt, datę, typ zdarzenia, tytuł, kwotę, walutę i informację, czy zdarzenie zwiększa saldo.
ProcessedOutboxMessage	Identyfikatory wiadomości Outbox już przetworzonych przez projekcję. Chroni przed wielokrotnym przetworzeniem tego samego zdarzenia.

9.3 Najważniejsze relacje

- Jeden klient ma wiele produktów.
- Produkt typu konto bankowe może mieć wiele transakcji.
- Przelew ma produkt źródłowy i opcjonalny produkt docelowy.
- Transakcja może być powiązana z przelewem.
- Jeden klient ma wiele kafli produktu i zdarzeń osi czasu w modelu odczytu.
- Rekord Outbox może odpowiadać jednemu rekordowi **ProcessedOutboxMessage**.

10 Frontend

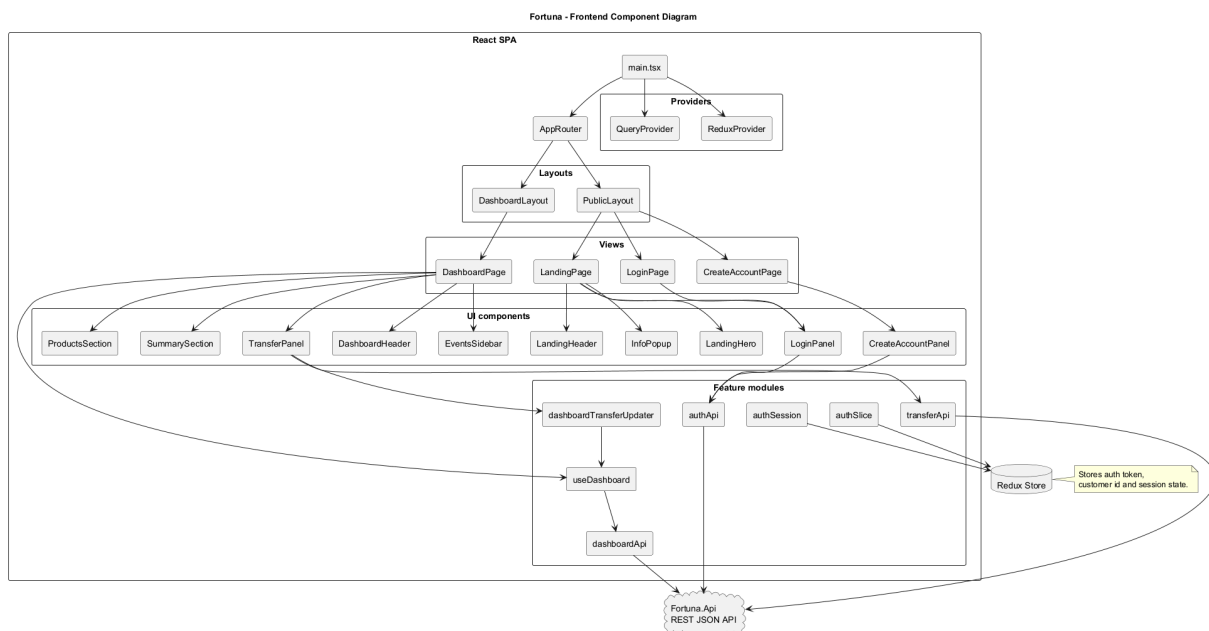
Frontend znajduje się w katalogu `frontend/src`. Aplikacja używa React Router do routingu, Redux Toolkit do stanu autoryzacji oraz TanStack Query do pobierania danych dashboardu.

10.1 Struktura

Katalog	Opis
<code>app</code>	Konfiguracja routera, klient API, providery i store Redux.
<code>features/auth</code>	Logowanie, rejestracja, sesja użytkownika i slice autoryzacji.
<code>features/dashboard</code>	Pobieranie danych dashboardu, typy i prezentacja produktów.
<code>features/transfers</code>	API przelewów, typy formularza i lokalna aktualizacja dashboardu po przelewie.
<code>components</code>	Komponenty widoków: landing page, logowanie, rejestracja, dashboard i przelewy.
<code>views</code>	Komponenty stron routingu: landing, login, create account, dashboard.
<code>shared/ui</code>	Wspólne komponenty UI, np. Button, Input, InfoPopup.

10.2 Komunikacja z API

Frontend używa funkcji `apiRequest`. Rejestracja i logowanie wysyłają żądania anonimowe. Dashboard i przelewy dodają nagłówek `Authorization`. Po wykonaniu przelewu frontend wykonuje lokalną, optymistyczną aktualizację danych, a następnie może pobrać odświeżony dashboard z read modelu.



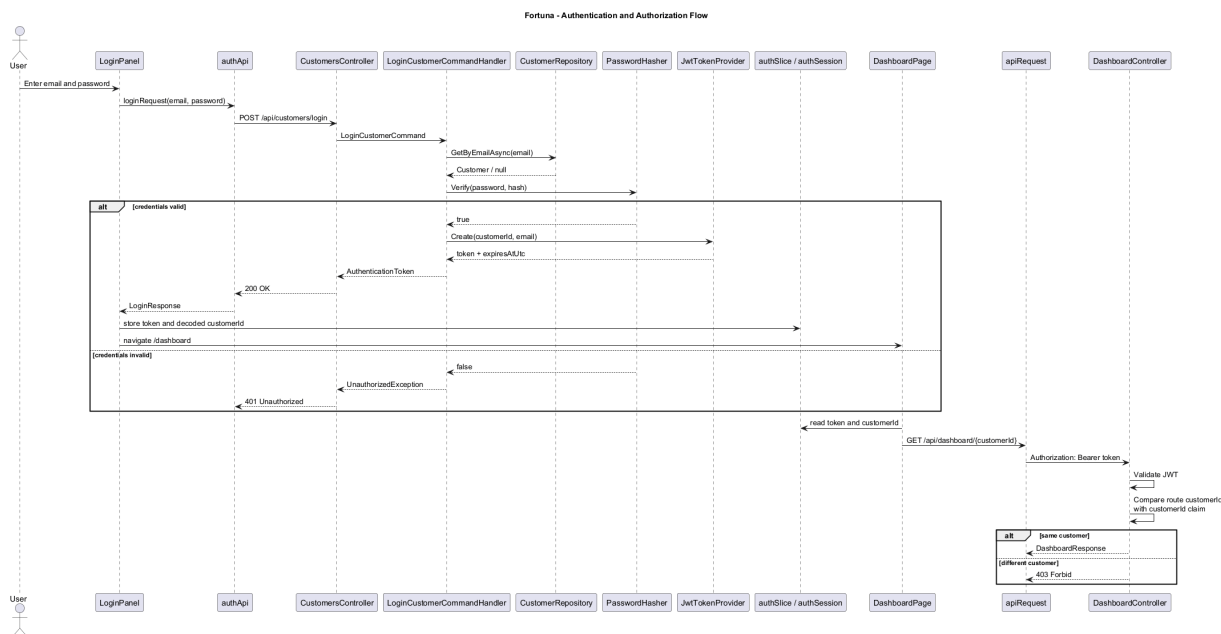
Rysunek 14: Diagram komponentów frontendu.

11 Bezpieczeństwo i walidacja

Backend używa JWT Bearer Authentication. Endpointy kont, dashboardu i przelewów wymagają zalogowania. Kontroler dashboardu dodatkowo sprawdza, czy klient próbuje pobrać własne dane, porównując identyfikator z URL z identyfikatorem z tokenu.

Hasła nie są przechowywane jawnie. Implementacja `Pbkdf2PasswordHasher` zapisuje hash hasła. Rejestracja wymaga hasła o długości co najmniej 8 znaków, a logowanie zwraca błąd autoryzacji dla niepoprawnego e-maila albo hasła.

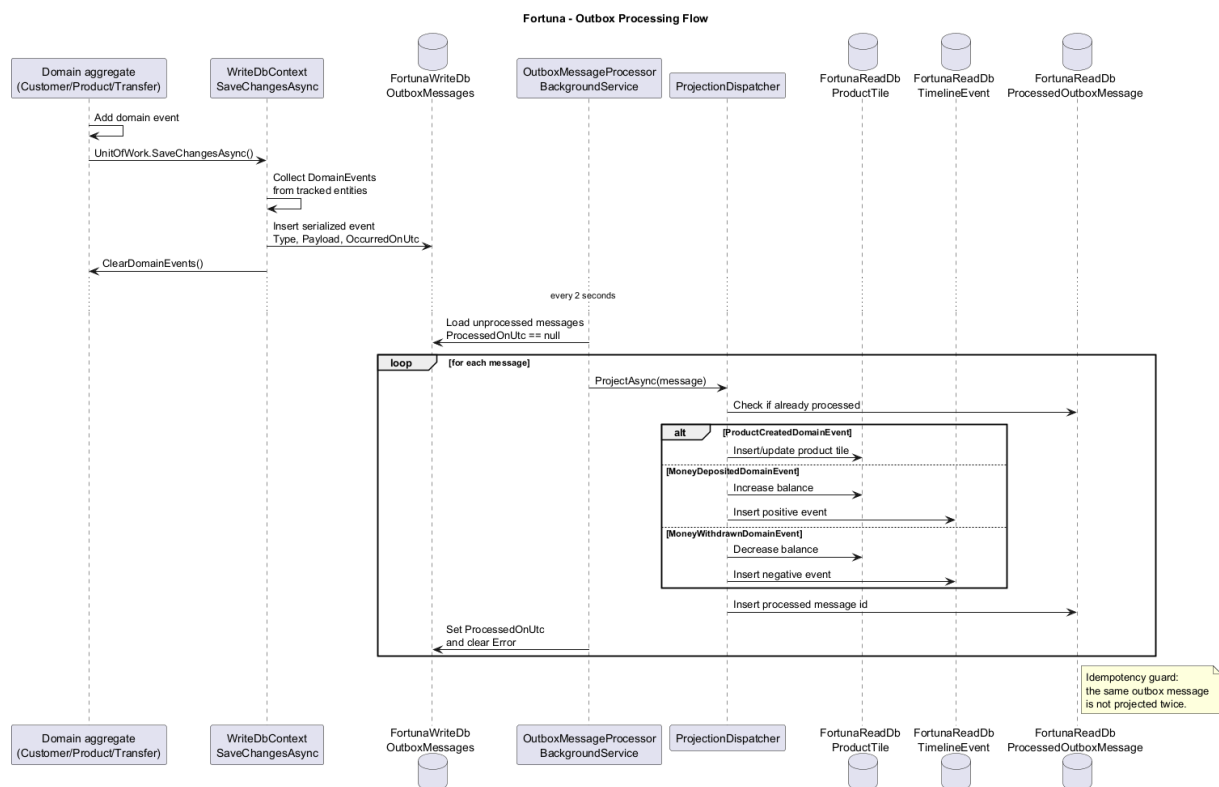
Walidacja reguł biznesowych znajduje się w handlerach i modelu domenowym. Przykładowo przelew zewnętrzny wymaga numeru rachunku i nazwy odbiorcy, przelew własny wymaga innego produktu docelowego niż źródłowy, a operacje wpłat i wypłat są obsługiwane tylko dla kont oraz kart.



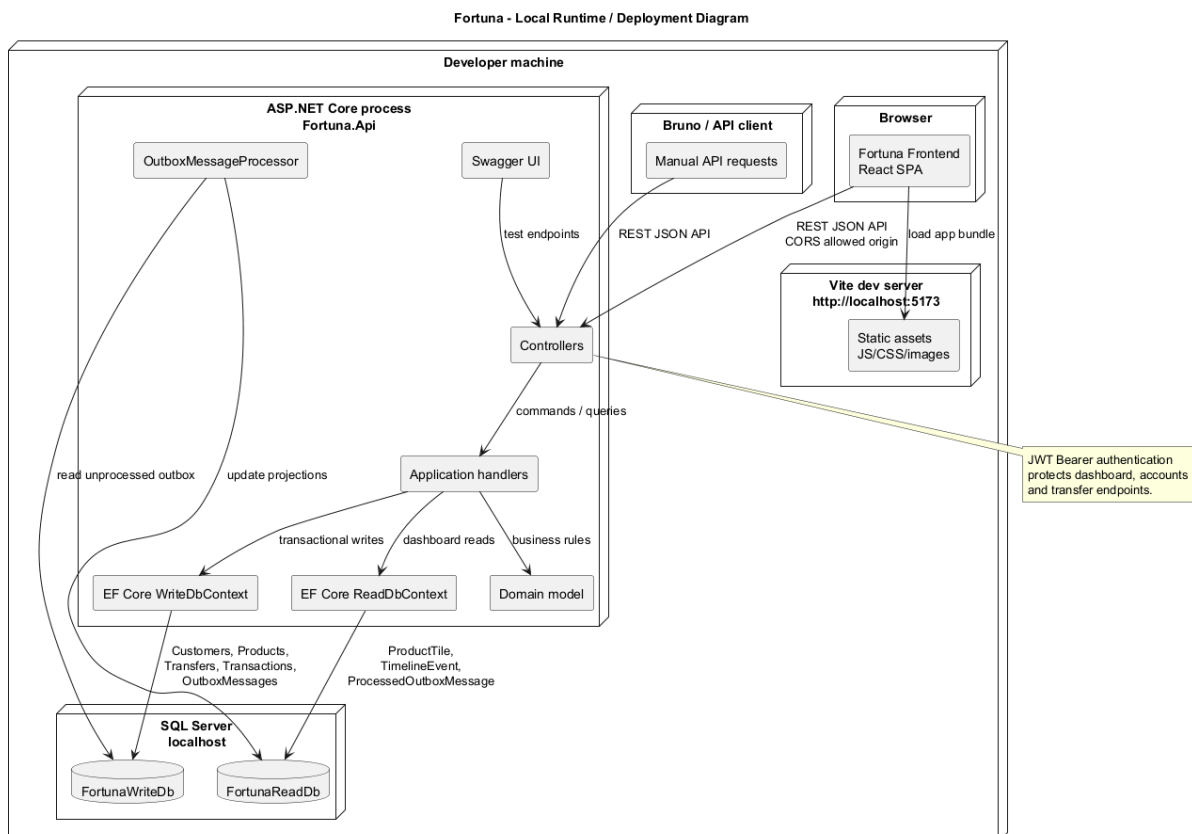
Rysunek 15: Przepływ uwierzytelniania i autoryzacji JWT.

12 Dodatkowe diagramy architektoniczne

Poniższe diagramy uzupełniają główne widoki UML o perspektywę uruchomieniową oraz o szczegóły asynchronicznej synchronizacji modelu odczytu.



Rysunek 16: Przetwarzanie zdarzeń domenowych przez mechanizm Outbox.



Rysunek 17: Lokalny diagram uruchomieniowy systemu.

13 Podsumowanie

Fortuna jest aplikacją wielowarstwową, w której frontend React komunikuje się z backendem ASP.NET Core przez REST API. Backend realizuje Clean Architecture i DDD, a przypadki użycia są rozdzielone zgodnie z CQRS. Model zapisu przechowuje pełny stan domenowy i generuje zdarzenia, natomiast osobny model odczytu udostępnia dane zoptymalizowane dla dashboardu. Mechanizm Outbox zapewnia asynchroniczne i kontrolowane przenoszenie zmian z modelu zapisu do modelu odczytu.